1. PARTIAL D. $f(x) \to f'(x)$: $x \to 1$ | const $\to 0$ | $\cos(x) \to -1 \cdot \sin(x)$ | $\ln(x) \to 1/x$

$\frac{\partial}{\partial x_3} x_2 = 0$ (var. not involved = const.)

linear combination $\frac{\partial}{\partial x_i}(k \cdot f(\vec{x}) + l \cdot g(\vec{x}) \dots) = k \cdot \frac{\partial f(\vec{x})}{\partial x_i} + l \cdot \frac{\partial g(\vec{x})}{\partial x_i}$

CR $\to (f \circ g)' = f'(g(x)) \cdot g'(x)$ | QR $\to (f/g)' = (f'g - fg')/g^2$ | PR $\to (fg)' = f'g + fg'$ | SR $\to (f+g)' = \alpha f' + \beta g'$ (CONST.)

GRADIENT DESCENT

$\vec{x}(i+1) = \vec{x}(i) - \varepsilon \cdot \vec{\nabla} f$ | $\vec{x}(1) = (0,1)^T - \varepsilon \cdot (f'x_1, f'x_2)^T$ # $\vec{x}_i$

$f(\vec{x}) = x_1^2 + 2x_2^2$ | $\vec{x}(0) = (0,1)$ | $\vec{\nabla} f = (2x_1, 4x_2)^T$ APPLY

CALCULATE DERIVATIVES if $g' = 1$ use → do for 0, 1, 2, then validate GD: put results into the function

KRONECKER DELTA $\frac{\partial x_a}{\partial x_b} = \delta x_{ab}$ | $\sum_i \delta_{ib} x_i = x_b$ (eliminate sums) | $\delta_{ab} = \begin{cases} 1 & a=b \\ 0 & a \neq b \end{cases}$ | $\delta_{ij}\delta_{jk} = \delta_{ik}$ | $a_j \delta_{jk} = a_k$

$\frac{\partial x_{ij}}{\partial x_{bc}} = \delta_{ib}\delta_{jc}$

SOFTMAX COMPUTATION $S_i(\vec{x}) = \frac{\exp(x_i)}{\sum_k \exp(x_k)}$ | $D = \exp(x_1) + \exp(x_2) \dots$ | $S_1(\vec{x}) = \frac{\exp(x_1)}{D}$ | $S_2(\vec{x}) = \frac{e^{x_2}}{D}$

$y = Wx + b \to y_i = \sum_n W_{ij} x_j + b_i$

DERIVATIVES $\frac{\partial}{\partial x_i} S_j = \frac{\partial S_j}{\partial x_i} = \delta_{ij} S_i - S_i \cdot S_j$ (SUBST. INDICES) | $i = j \to \delta_{ij} = 1$ | $i \neq j \to \delta_{ij} = 0$

$0 < S_i < 1$

2. NUMPY & MATPLOTLIB

ARRAY CREATION: NP.ARANGE(0,101,2) | NP.LINSPACE(0,100,51) | NP.ARRAY([[1,1],[2,2]]) | NP.EYE(K) # IDENTITY | NP.ONES([3,5]) * 55 # 3×5 all 55 | NP.RANDOM.UNIFORM(0,1,[5,4,3])

SLICING: a[999] # 1000 0-INDEX | a[::2, :] row col | a[:, ::2] # every 2nd col | a[::-1, ::-1] # reverse both | a[::-2, ::-2] # reverse + skip | a[:5, :] # 1st 5 rows | a[:, :5] # last 5 col

MASK + FANCY INDEXING: v[v<10] # mask x<10 | X[X<0] = 0 # ReLU | v[[1,5,12]] # FANCY, SET of index | train[[0,4,18]] # EXTRACT SAMPLES | MASK = (train.argmax(axis=1) == 0 # class 0 mask | train[mask] # filter

BROADCASTING: V = NP.ARANGE(1,29).reshape(1,-1,1) # (1,29,1) add to ROWS | V = NP.ARANGE(1,29).reshape(1,1,-1) # (1,1,29) add to COLS

MATPLOTLIB: f, ax = plt.subplots(1,4) → x = NP.LINSPACE(1,5,100) → ax[0].plot(x, 1/x) # LINE | ax[1].scatter(x, 1/x) → ax[2].bar(x, 1/x) → ax[3].imshow(NP.RANDOM.UNIFORM(0,1,[20,20])

ADVANCED NUMPY: # ONE-HOT T = NP.EYE(K)[labels] | CM0[NP.ARANGE(N), t_hat, y_hat] += 1 | plt.show() | CONFUSION MATRIX CM0 = NP.ZEROS([N,K,K]) | CM = CM0.sum(axis=0)

SHUFFLE IN SYNC idx = NP.RANDOM.PERMUTATION(N) → X, T = X[idx], T[idx]

3. TF/KERAS FRAMEWORK

ReLU $R = NP.MAXIMUM(X, 0)$

CROSS ENTROPY $L = -(NP.LOG(Y + 1e-9) * T).sum(axis=1).mean()$

SOFTMAX $e = NP.EXP(X - NP.max(X, axis=1, keepdims=True)) \to S = e / NP.sum(e, axis=1, keepdims=True)$

NUMPY to TF translation: np.max → tf.reduce_max | NP.SUM → tf.reduce_sum | NP.MAXIMUM → tf.maximum | np.exp → tf.exp | np.dot → tf.matmul | keepdims = True → both

GRADIENT TAPE

```
x = tf.Variable([1., 2., 3.])
WITH tf.GRADIENTTAPE() as tape:
    y = tf.reduce_sum(x**2)
GRAD = TAPE.GRADIENT(Y, x)
PRINT(GRAD)
```

CM Probability from Tensor

```
CM = tf.constant([[10,60,10],[0,90,0],[0,20,60]], dtype=tf.float32)
row_prob = CM[i,j] / tf.reduce_sum(CM[i,:])
col_prob = CM[i,j] / tf.reduce_sum(CM[:,j])
```

$p(y=j \mid t=i)$

$p(t=i \mid y=j)$